KAFKA

1. What is Kafka?

- **Kafka is a high-throughput distributed event streaming platform.**
- **It is widely used in the industry – over 80% of Fortune 100 companies use Kafka.**
- **Examples of its heavy usage:**
  - **Manufacturing: 9/10**

  companies ○ **Banking: 7/10** ○

  Insurance: 10/10 ○

  Telecom: 8/10

2. Why Kafka? (Real-World Problem)

Example: Zomato / Food Delivery App

- **When you place an order, you see your delivery partner's live location on a map.**
- **The driver's phone sends location updates every second to the server.**
- **Server stores this location data and also shows it to the customer in real-time.**
- **Analytics are performed later (e.g., how much time delivery took, total distance, etc.).**

Basic Approach:

- **Every second, driver's phone → gets current location → sends it to server → stores in database → updates the customer.**

Problem with Traditional Databases:

- **Thousands of delivery partners = thousands of insert operations per second.**
- **Example: 1000 drivers sending data each second = 1000 DB writes/sec.**
- **SQL databases have limited throughput (can't handle so many write operations).**
- **Eventually, the database will crash or slow down.**

Technical Terms:

- **Throughput: Number of operations per second the system can handle.**
- **SQL DBs have low throughput but high storage (can store huge amounts of data).**
- **Kafka has high throughput, but low long-term storage capacity.**

---

3. More Real-World Examples

a. Discord (or any chat app):

- **Thousands of users chatting simultaneously.**
- **Each message needs to be stored and then broadcasted to others.**
- **Storing each message instantly in DB leads to delay and high load.**
- **Results in:**
  - **Not real-time experience (e.g., 1 second delay)**
  - **DB may crash under high load.**

b. Uber/Ola:

- **Driver location, speed, ride data sent every second.**
- **Multiple services need that data: fare calculation, analytics, customer view, etc.**
- **Each service performs different DB operations.**
- **Leads to 3000+ DB operations per second per 1000 drivers.**
- **DBs cannot scale to this extent.**

---

4. Solution: Kafka

Kafka's Superpower:

- **Kafka has very high throughput.**
  - **Can handle millions of messages per second.**
  - **Can produce and consume data at very high speed.**
- **Kafka is not a replacement for databases – it is used with them.**

Difference Between Kafka and Database:

Feature	Kafka	Database
Throughput	Very High	Low
Storage	Temporary (short-term)	Long-term persistent
Query Support	No real querying	Supports complex queries

---

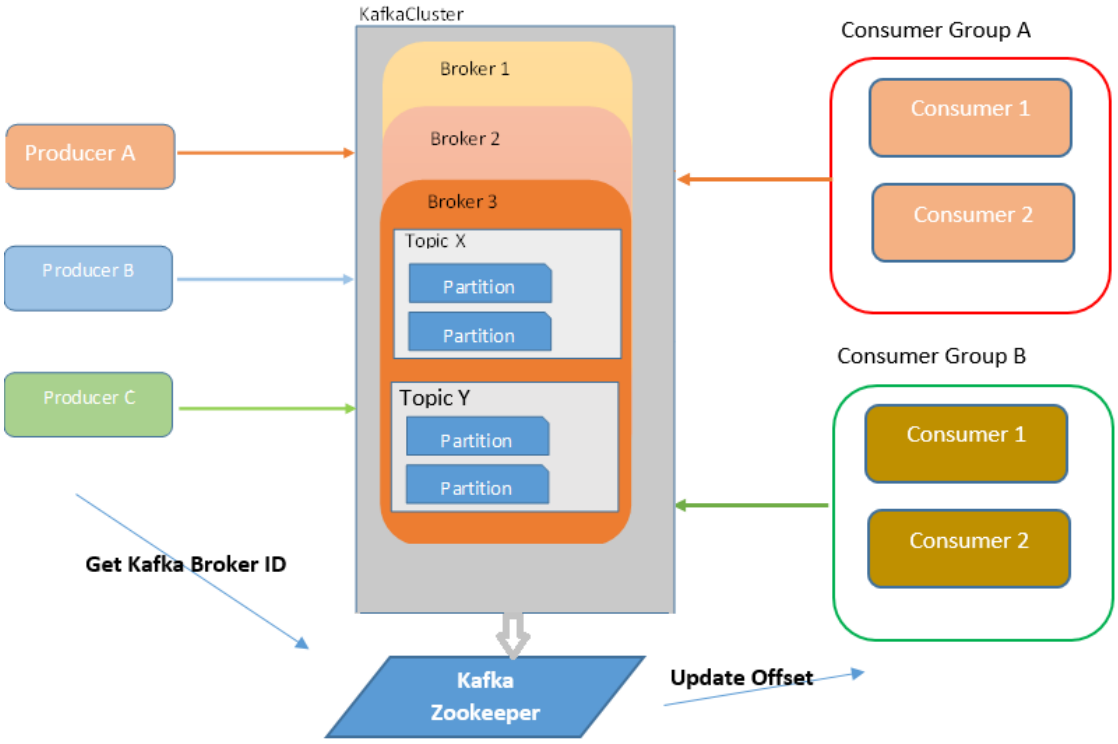
5. Kafka Architecture (Simplified)

- **Producers: Send data to Kafka (e.g., drivers sending location).**
- **Kafka: Acts as a buffer or queue system – temporarily stores and streams data.**
- **Consumers: Pull data from Kafka and process it.**

- Can be different services: analytics, customer display, fare calculator, etc.

Kafka in Action:

- **Instead of storing each location instantly in DB:**
  1. **All drivers send data to Kafka.**
  2. **Consumers process that data in bulk (e.g., batch every 2 seconds).**
  3. **Final processed data is inserted into DB in one single operation (bulk insert).**



Source: TutorialsPedia.com

---

6. Kafka Topic

- **Topics in Kafka = Channels or categories of messages.**
- **Example:**
  - **Topic 1: Rider location updates**
  - **Topic 2: Hotel updates**
- **Each topic is further divided into partitions for scalability.**

---

7. Conclusion

- **Kafka solves real-time data ingestion problems.**
- **Kafka is not a database, but complements it.**

- **It acts as a middle layer for high-speed data transfer and aggregation before database insertion.**
- **Ideal for real-time systems like food delivery apps, ride-hailing, chat platforms, etc.**

Kafka Topic & Partitions Setup

1. **Multiple Topics** ○

   Example topics:

   - ✦ **rider_updates: All delivery partner updates**
   - ✦ **hotel_updates: Status of hotel orders**

2. **Partitioning the Topic**

   - ○ **Instead of dumping all messages into one stream, Kafka divides topics into partitions.**
   - ○ **Partitioning improves parallelism and fault isolation.** ○ **Similar to database sharding: splitting data across multiple logical units for performance.**

3. **Partitioning Strategy** ○

   Can partition

   by:

   - ✦ **Location (e.g., North vs South India)**
   - ✦ **Parameters like usernames, geographies, etc.**

   - ○ **Ensure consistent partitioning logic, not time-based splits.**

4. **Consumer Isolation** ○ **Each partition is**

   consumed independently.

   - ○ **Different services/processes can read from specific partitions.**

Kafka Internals: Topic and Partition Behavior

1. **Topic with Partitions** ○ **A topic is split into numbered**

   partitions (e.g., 0, 1, 2, 3).

2. **Producer Logic** ○ **Producers send messages to Kafka and**

   choose which partition they belong in. ○ **Custom logic**

   (partitioner) ensures correct assignment (e.g., South India →

   partition 0; North India → partition 1).

3. **Consumers & Auto-Balancing** ○ **A consumer group contains one or more consumers.**
    - **Kafka automatically balances partitions among consumers:**
        - **1 consumer = reads all partitions.**
        - **2 consumers = each reads half.**
        - **Additional consumers beyond partitions stay idle—no partition assigned.**
    - **Constraint: One partition can be handled by only one consumer within the same group.**
4. **Consumer Groups** ○ **Consumer grouping enables scaling and parallelism.** ○ **Each consumer group is an independent stream.** ○ **Multiple consumer groups can subscribe to the same topic, allowing different services to process the same data.**

Message Delivery Models

1. **'Queue' Behavior (1-to-1)** ○ **One producer, one consumer – traditional queue pattern (First-in, First-out).**
2. **'Pub-Sub' Behavior (1-to-Many)**
    - **One producer, multiple consumers (in different consumer groups).**
    - **Kafka supports both patterns via consumer groups and partitions.**

Kafka Development Environment Setup

1. **Prerequisites** ○ **Intermediate Node.js knowledge** ○ **Basic distributed systems understanding** ○ **Docker installed**
2. **Setting Up ZooKeeper & Kafka via Docker** ○ **Launch ZooKeeper (default port 2181)** ○ **Launch Kafka broker (default port 9092)**
    - **Use the official Confluent Kafka Docker image**
3. **Code Architecture Components** ○ **Admin: Creates infrastructure – topics and partitions** ○

Producer: Sends messages to Kafka ○ Consumer:

Reads messages from Kafka

4. Sample Node.js Project ○ Create admin.js,

producer.js, consumer.js

   - Use KafkaJS or similar library to:
     - Define Kafka client connecting to broker
     - Create topic with partitions and replication factor
     - Implement producer/consumer logic

Summary

- Kafka topics are partitioned for scalability and parallelism.
- Producers assign messages to partitions based on logic.
- Consumers in a group share partitions automatically.
- Supports both queue and pub-sub message models.
- Setup involves ZooKeeper + Kafka, managed through Docker.
- Node.js with KafkaJS is used to handle admin, producer, and consumer tasks.

1. Kafka Admin Setup

Define Kafka Broker

- The Kafka broker is hosted at localhost:9092 or your machine's IP address.
- Port 9092 is the Kafka default.
- Only one broker is being used here.

Create Kafka Client

- A Kafka client is created using a clientId.
- The client ID can be any string (e.g., "my-app").

Create Admin

js

CopyEdit const admin =

kafka.admin(); await

admin.connect();

Create Topic

- A topic called "rider-updates" is created.
- The topic has 2 partitions:
  - partition 0: for North India
  - partition 1: for South India
- The creation is logged to the console.

---

2. Kafka Producer Setup

Create Producer Instance

js

CopyEdit const producer =

kafka.producer();

await producer.connect();

- Logs “Producer connected successfully”.

Produce a Message

- A message is sent to topic "rider-updates":

js

CopyEdit topic: "rider-

updates", messages: [

{

key: "rider", // e.g., "tony" value: JSON.stringify({ name:

"Tony Stark", location: "south" }), partition: 1 // since location

is south

}

]

- Messages are hardcoded initially.
- Later enhanced to accept user input using readline.

---

3. User Input for Message Production

Using readline for Dynamic Input

- The app asks:
  - Rider name (e.g., "Thor") ○ Location ("north" or "south")

Logic for Partition Assignment const partition =

location.toLowerCase() === "north" ? 0 : 1;

- **Messages are produced based on user input.**

Graceful Shutdown

process.on("close", async () => {

await producer.disconnect();

});

---

4. Kafka Consumer Setup

Create Consumer const consumer = kafka.consumer({

groupId: "user-1" }); await consumer.connect();

Subscribe to Topic

await consumer.subscribe({ topic: "rider-updates", fromBeginning: true });

Run Consumer

await consumer.run({ eachMessage: async ({ topic,

partition, message }) => { console.log("New

message"); console.log(`Topic: ${topic}`);

console.log(`Partition: ${partition}`);

console.log(`Message: ${message.value.toString()}`);

},

});

---

5. Consumer Groups & Partition Balancing

One Group, One Consumer

- **One consumer in group user-1 reads all partitions.**

One Group, Two Consumers

- **Kafka auto-balances partitions:**
  - **One consumer gets partition 0 ○ Second gets partition 1**

Different Groups

- Two consumer groups (e.g., user-1, user-2) both receive all messages.
    - This is the publish-subscribe pattern.

---

6. Kafka Concepts Illustrated

Concept	Example
Broker	Kafka running on localhost:9092
Topic	rider-updates
Partitions	0 = north, 1 = south
Producer	Sends rider name & location
Consumer Group	user-1, user-2
Concept	Example
Partition Assignment	Auto-balanced within a group
Pub-Sub	Same message sent to multiple groups

Final Output Example

If "Thor north" is entered:

- Producer sends to partition 0
- Consumer in user-1 or user-2 logs:

plaintext

CopyEdit

New message

Topic: rider-updates

Partition: 0

Message: {"name":"Thor", "location":"north"}

Summary

- Admin creates topics and partitions.
- Producer dynamically sends messages based on user input.
- Consumers listen to assigned partitions via group management.

- **Kafka auto-balances consumers in the same group.**
- **Pub-Sub supported via different groups.**